

PROJECT INTERVIEW DOSSIER · 30 SEPTEMBER 2026

Postmark

Fake Job Posting Screening Application

A complete, code-grounded preparation guide for explaining the project on a resume, in a technical interview, and in a behavioral/project discussion.

Python

Streamlit

pandas

scikit-learn

SQLite

NLP

One-line explanation: Postmark is a local Streamlit application that combines job-posting text fields, converts them into TF-IDF features, compares Logistic Regression, Multinomial Naive Bayes, and a soft-voting ensemble, and presents a recall-aware fraud-screening signal while storing only a minimal screening summary.

This guide describes the implementation that exists in the repository. It deliberately distinguishes implemented behavior from recommended future improvements.

1. Executive summary

Postmark addresses a practical trust-and-safety problem: job seekers often need a fast second opinion about language associated with employment scams. The application accepts a job posting, evaluates its text with a supervised binary classifier, and gives a human-readable screening result. It is a screening aid, not an autonomous decision-maker.

What the application does

- Loads a labeled CSV or ZIP dataset from a configured path, project folder, Downloads folder, or an in-memory synthetic fallback.
- Accepts common label-column names and normalizes numeric or textual labels into genuine (0) and fraudulent (1).
- Combines relevant job fields into one labeled text string, preserving field names such as `title:` and `requirements:`.
- Uses TF-IDF with English stop-word removal, unigrams and bigrams, sublinear term frequency, and a 30,000-feature cap.
- Compares Logistic Regression, Multinomial Naive Bayes, and a weighted soft-voting ensemble.
- Evaluates on a stratified 75/25 hold-out split using accuracy, precision, recall, and F1.
- Refits each model on all labeled rows after comparison, so all three are available for screening.
- Shows a risk label, model score, separate phrase cues, model metrics, dataset information, and recent screening history.
- Stores only title, result, score, model name, and timestamp in SQLite; the original job description is not saved.

Important honesty point: The score is a classifier output, not a calibrated probability. If the synthetic fallback is active, its metrics are only for walkthrough behavior and must not be presented as production evidence.

2. Repository map

File	Responsibility	How to discuss it
<code>app.py</code>	Streamlit UI, session state, training triggers, prediction flow, safety messaging, metrics and history display.	The orchestration layer: it connects user actions to loader, detector and storage modules.
<code>detector.py</code>	Text composition, label detection/normalization, model construction, evaluation, full-data refit and prediction.	The ML/domain layer. This is the most important file for technical discussion.

dataset_loader.py	CSV/ZIP ingestion, archive member selection, file-size validation and default dataset discovery.	The input boundary, including practical validation and safe in-memory ZIP reading.
demo_data.py	Creates synthetic walkthrough rows when no local labeled data is available.	A development/demo fallback, explicitly not a benchmark.
storage.py	SQLite schema and parameterized insert/query functions for recent screening summaries.	A small persistence layer designed around data minimization.
tests/test_detector.py	Six unittest cases covering text handling, labels, training, prediction, storage privacy and ZIP loading.	Evidence that the core behavior is testable independently of the UI.
requirements.txt	Version ranges for pandas, scikit-learn and Streamlit.	Defines the runtime dependency contract.
README.md	Setup commands, data format, limitations and usage instructions.	The operational documentation and responsible-use contract.
postmark.db	Local SQLite file created/used at runtime.	Runtime state, not application logic; it is ignored by .gitignore.

There is no separate frontend framework, REST API, authentication system, cloud service, model registry, migration system, Dockerfile, or deployment configuration in the current repository. Streamlit renders the interface directly from Python.

3. Architecture and runtime flow

```

User opens Streamlit
  |
  v
app.py initializes SQLite and session state
  |
  +--> dataset_loader.py finds CSV/ZIP
      |      (or demo_data.py creates synthetic data)
      |
      +--> detector.train_and_compare(frame)
          |      1. find/normalize labels
          |      2. compose text
          |      3. stratified split
          |      4. TF-IDF fit on train only
          |      5. train/evaluate 3 models
          |      6. refit models on all rows
          |
          v
Sidebar selects model; form collects job fields
  |
  v
detector.predict_post(selected_model, record)

```

```
|  
+--> app displays label, score and independent phrase cues  
+--> storage.save_prediction(summary only)  
|  
v  
Model lab displays metrics, source and recent SQLite history
```

Why this structure is effective

- UI code does not contain model mathematics.
- Data loading is isolated from training, so local files and uploads share validation rules.
- Core ML functions accept pandas records/dataframes and can be tested without starting Streamlit.
- Persistence is isolated, making the privacy policy visible and testable.
- The design is small enough for a portfolio project but leaves clear seams for production hardening.

4. Startup and user journey

1. **Startup:** importing `app.py` calls `initialize_database()`, then configures the Streamlit page.
2. **Initial training:** `MODEL_SOURCE_VERSION` is checked in session state. When the version is new for the session, a default dataset is loaded or synthetic data is built and `train_and_compare` runs.
3. **Sidebar:** the user can upload CSV/ZIP data, train from it, select one of the three classifiers, or reload the default source.
4. **Screening form:** the user enters title, company profile, location, description, requirements, benefits and salary range. Sample buttons populate two synthetic examples.
5. **Prediction:** the selected model receives the same field-composition logic used during training. Empty text is rejected explicitly.
6. **Presentation:** the UI shows a binary category, a score bar, an uncertainty band for scores from 0.4 through 0.6, and fixed phrase-cue checks.
7. **Persistence:** the title and result summary are inserted into SQLite. Full text is intentionally omitted.
8. **Model lab:** metrics and recent history are shown, and the synthetic walkthrough CSV can be downloaded.

5. Data ingestion in detail

Default search order

1. `POSTMARK_DATASET_PATH`, if set and the path exists.
2. `fake_job_postings.csv` beside `app.py`.
3. `~/Downloads/archive (4).zip`.
4. No dataset: synthetic walkthrough corpus.

CSV and ZIP behavior

Both filesystem and uploaded bytes are supported. ZIP archives are read without extracting members to disk. The loader selects `fake_job_postings.csv` when present; otherwise it accepts a ZIP with exactly one CSV and rejects ambiguous archives. CSV content is capped at 200 MB. A malformed uploaded ZIP raises a clear validation error.

Labels

Accepted label-column names are `fraudulent`, `is_fraudulent`, `label`, `class`, and `target`. Matching is case-insensitive after trimming column names. Positive values include 1, true, yes, fake, fraud, fraudulent and scam. Negative values include 0, false, no, real, legitimate, genuine and not fraudulent. At least four examples of each class are required.

The minimum-class rule is intentionally conservative: a train/test split with too few examples can become unstable or fail to represent both classes. It is a usability guard, not a guarantee of statistical quality.

6. Text representation and feature engineering

`compose_text` iterates over a fixed set of fields: title, location, department, salary range, company profile, description, requirements, benefits, employment type, required experience, required education, industry and function. Missing values and blank strings are skipped.

```
title: remote payment processing agent location: remote
description: earn thousands weekly no interview
requirements: send bank details immediately
```

Field names are included as tokens. This preserves a small amount of structural information without building separate feature pipelines for every column. For example, a word in a requirement field can remain distinguishable from a word in a company profile field.

TF-IDF settings

Setting	Value	Reason
stop_words	english	Removes common words with limited discriminative value.
ngram_range	(1, 2)	Uses individual words and two-word phrases such as “bank details” or “no experience”.
max_features	30,000	Caps memory and computation while retaining a useful vocabulary.
min_df	1	Keeps rare terms, useful for small datasets but potentially noisier.
sublinear_tf	True	Uses a logarithmic term-frequency effect so repetition does not dominate linearly.

Interview explanation: TF-IDF is a strong baseline here because job posts are sparse text, training is fast, the feature space is understandable, and it works well with linear models. It is more practical

for this project than a large language model because the goal is a lightweight local screening tool.

7. Models, evaluation and selection

Logistic Regression

Logistic Regression is a linear classifier over the sparse TF-IDF vector. `class_weight="balanced"` reduces the tendency to ignore the minority class when the dataset is imbalanced. `max_iter=1000` gives optimization enough iterations, and `random_state=42` makes the configuration reproducible.

Multinomial Naive Bayes

Multinomial Naive Bayes is a classic fast text baseline. It works with non-negative TF-IDF values and uses `alpha=0.5` smoothing to avoid zero-probability effects for unseen features.

Soft-voting ensemble

The ensemble combines the two estimators using probability outputs with weights 2:1 in favor of Logistic Regression. This can smooth individual model behavior, but the weights are a design choice that should be validated rather than assumed to be universally optimal.

Evaluation sequence

1. Drop rows with missing labels.
2. Normalize labels.
3. Split text and labels with `test_size=0.25`, `random_state=42` and `stratify=labels`.
4. Fit the vectorizer only on training text, then transform the test text. This prevents test vocabulary from influencing training.
5. Fit all three classifiers on training vectors.
6. Compute accuracy, precision, recall and F1 on the held-out vectors.
7. Sort the displayed metric table by F1.
8. Clone the vectorizer, fit it on all text, and refit every model on all labels for final screening.

Best versus recommended model

`best_model` is the model with the highest hold-out F1. `recommended_model` is selected by highest recall and then highest precision. This is deliberate: missing a fraudulent post can be more harmful than sending a legitimate post for human review. The UI therefore defaults to a recall-oriented choice rather than blindly selecting the highest F1.

8. Prediction behavior and safety UX

`predict_post` composes text, rejects an entirely empty record, calls `predict_proba`, locates the probability for class 1 by reading `model.classes_`, and returns a binary label at a threshold of 0.5 plus the fraud-class

score.

The UI uses three presentation bands: scores from 0.4 to 0.6 are “Elevated risk” and visually marked for review; outside that band, a positive result is “Higher risk” and a negative result is “Lower risk.” This is a communication choice, not a change to the classifier threshold.

Phrase cues are a separate deterministic checklist for phrases involving upfront payment, sensitive financial details, unusually high pay, urgency and avoiding an interview. The app explicitly says these cues are not an explanation of the model decision. This distinction matters because a post can receive a high model score without containing one of the exact hard-coded phrases.

Responsible-use boundary: The model must not be the sole basis for rejecting an employer or accusing a person of fraud. Text-only classification can miss scams, flag legitimate posts, and become stale as scam wording changes. Independent verification is required.

9. Persistence and privacy

SQLite is used because this is a local, lightweight, single-user-style application. The table is created if missing:

```
predictions(  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  job_title TEXT NOT NULL,  
  is_fraudulent INTEGER NOT NULL CHECK (is_fraudulent IN (0, 1)),  
  fraud_score REAL NOT NULL CHECK (fraud_score >= 0 AND fraud_score <= 1),  
  model_name TEXT NOT NULL,  
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
)
```

Writes use parameterized SQL, and the recent-history query clamps the requested limit between 1 and 100. The code stores no description, requirements, company profile, or other source text. The test explicitly verifies that the returned history does not include a description.

Data minimization reduces exposure, but it is not full security: a local SQLite file can still be read by anyone with filesystem access, titles may contain personal information, and there is no authentication or encryption in this version.

10. Testing and verification

Test	What it proves
Empty fields in <code>compose_text</code>	Missing values do not create junk tokens.
Case-insensitive label discovery	Dataset column capitalization does not break ingestion.
Three-model training and prediction	All expected models, metrics columns, split counts and prediction range work together.

Insufficient/single-class data	Training fails clearly instead of producing an unusable classifier.
SQLite privacy summary	Results persist and raw description content is not returned.
CSV inside ZIP	Both filesystem and upload loaders read the same archive format.

Verification command:

```
python -m unittest discover -s tests -v
```

Current verification result: all 6 tests pass. The tests do not yet cover Streamlit browser interactions, every label alias, malformed CSV variants, threshold tuning, cross-validation, or production-scale performance.

11. How to run and demo it

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

1. Open the local Streamlit URL.
2. Explain that the app may use synthetic walkthrough data if no labeled dataset is found.
3. Click “Load established role,” analyze it, and show the lower-risk result.
4. Click “Load suspicious offer,” analyze it, and show the elevated/higher-risk result and phrase cues.
5. Open Model lab to explain hold-out metrics, the data source and the privacy-preserving history table.
6. If available, upload the labeled Kaggle CSV/ZIP and explain that the models are retrained and refit.

12. Resume-ready wording

Short project entry

Postmark — Fake Job Posting Screening | Python, Streamlit, scikit-learn, pandas, SQLite

Built a local NLP screening application that combines job-post fields, extracts TF-IDF uni/bi-gram features, compares Logistic Regression, Multinomial Naive Bayes and a weighted soft-voting ensemble, and evaluates them with stratified hold-out accuracy, precision, recall and F1. Added CSV/ZIP ingestion validation, recall-oriented model recommendation, human-readable risk cues, and privacy-conscious SQLite history that excludes full job-post text.

30-second pitch

“I built Postmark to give job seekers a second opinion on suspicious job-post language. The app is a Streamlit front end over a small scikit-learn pipeline. It normalizes common labeled datasets, combines relevant fields into text, uses TF-IDF with unigrams and bigrams, and compares two complementary classifiers plus a soft-voting ensemble. I use stratification because fraud data can be imbalanced, report

precision/recall/F1, and default toward recall because missing a scam is costly. I also separated deterministic phrase cues from the model output so I do not pretend they explain the classifier. Screening history is stored in SQLite without the original description.”

Do not claim

- Do not claim production-grade accuracy unless you have measured it on a real, representative dataset.
- Do not call the score a probability or the app an automated fraud adjudicator.
- Do not claim an API, authentication, deep learning, explainable AI, or cloud deployment; those are not in this repository.

13. Interview questions and strong answers

Q1. What problem does this project solve?

It screens job-posting language for signals associated with employment scams, giving a human reviewer a fast second opinion. It is intentionally a screening aid rather than a final decision system.

Q2. Walk me through the architecture.

Streamlit provides the UI and orchestration. `dataset_loader.py` validates and reads CSV/ZIP input. `detector.py` handles text composition, labels, vectorization, training, evaluation and prediction. `storage.py` initializes SQLite and stores only a summary. `demo_data.py` provides a synthetic fallback. Core logic is tested independently in `tests/test_detector.py`.

Q3. Why did you choose TF-IDF?

The input is sparse, natural-language text and the project needs a fast, local, explainable baseline. TF-IDF is computationally inexpensive, works well with linear text classifiers, and lets us capture both individual words and short phrases without requiring a large model.

Q4. Why include bigrams?

Many scam cues are multi-word expressions, such as “bank details,” “no experience,” or “pay a fee.” Bigrams capture some phrase meaning that unigrams lose while keeping the feature space manageable.

Q5. Why compare Logistic Regression and Naive Bayes?

They are strong, fast and complementary text baselines. Logistic Regression learns a discriminative linear boundary and supports class balancing. Naive Bayes is simple and often competitive on word-frequency-style text problems. Comparing them is more defensible than assuming one model is best.

Q6. Why use an ensemble?

The soft-voting ensemble combines their probability outputs. The current implementation weights Logistic Regression twice as heavily as Naive Bayes. It can be more stable than either model alone, but I would validate the weights with cross-validation rather than treat 2:1 as universally correct.

Q7. Why is the split stratified?

Fraudulent posts may be the minority class. Stratification preserves the class ratio in train and test sets so the test set does not accidentally omit or underrepresent a class.

Q8. Why use a fixed random seed?

A fixed seed makes the hold-out comparison reproducible during development and demos. In a stronger evaluation process I would also use repeated stratified cross-validation and report variation, not just one split.

Q9. What is data leakage and how do you avoid it?

Leakage occurs when information from the test set influences training. The vectorizer is fit only on the training text before test transformation. Only after evaluation do we create a separate full-data vectorizer and refit final screening models on all labeled rows.

Q10. Which metric matters most?

It depends on the cost of errors. Recall matters because missing a fraudulent post is risky; precision matters because too many legitimate posts sent for review reduce trust; F1 balances both. The UI displays all four and recommends a model by recall first, then precision, while ranking the comparison table by F1.

Q11. Why not optimize accuracy?

Accuracy can look good on imbalanced data even when the minority class is handled poorly. A fraud-screening workflow needs class-specific metrics, especially recall and precision, rather than a single overall correctness number.

Q12. Is the score a probability?

No. It is the model's class-1 probability-like output and is shown as a model score. It is not calibrated evidence that the listing has that exact chance of being fraudulent. Calibration curves, Platt scaling or isotonic calibration would be future work.

Q13. How does the application handle missing fields?

The composer skips null and blank values. The prediction path rejects a record only when the complete composed text is empty, so users can analyze a post with only some fields populated.

Q14. What happens with an invalid dataset?

The loader rejects unsupported extensions, oversized files, malformed ZIPs and ambiguous multi-CSV archives. The detector raises clear errors for missing labels, empty labels, unsupported label values and insufficient examples of either class. The Streamlit UI catches expected validation errors and displays them.

Q15. Why store only a summary?

The job description may contain personal or sensitive information. Storing only title, result, score, model and timestamp follows data minimization and limits unnecessary retention. It does not eliminate all privacy risk because titles and local database access still need protection.

Q16. Is the database secure?

It is lightweight local persistence, not a hardened security system. SQL writes are parameterized and constraints protect value ranges, but there is no authentication, encryption or multi-user isolation. I would address those before deploying it for sensitive or shared use.

Q17. Why Streamlit?

Streamlit enabled a usable data/ML interface quickly while keeping the portfolio project in Python. It is suitable for a local prototype. For a multi-user production product, I would separate a frontend, API, model service and managed database.

Q18. How does Streamlit state work here?

Streamlit reruns the script when controls change, so trained results, form values and the latest prediction are kept in `st.session_state`. `MODEL_SOURCE_VERSION` prevents default retraining on every rerun during a session; uploading or reloading a dataset explicitly replaces the stored training result.

Q19. What are the main limitations?

It is text-only, wording can drift, the score is not calibrated, small hold-out sets can be unstable, the synthetic fallback is not performance evidence, there is no authentication, and the current test suite does not cover UI behavior or production monitoring.

Q20. How would you improve evaluation?

Use a representative real dataset, remove duplicates, perform repeated stratified cross-validation, report confidence intervals, inspect the confusion matrix and false negatives, test temporal generalization, tune the threshold against explicit costs, calibrate scores, and keep a final untouched test set.

Q21. How would you improve the ML model?

Start with better labels and leakage checks. Then compare character n-grams for obfuscated wording, calibrated linear models, threshold tuning, class-specific error analysis, and possibly a transformer embedding model if latency, privacy and data volume justify it. I would keep the simple baseline for comparison.

Q22. How would you deploy it?

Package dependencies, pin a tested environment, put the model artifact behind a versioned service, move history to a managed database, add authentication and authorization, protect uploads, add monitoring for drift and false negatives, and make the model/data version visible in each result. Streamlit can remain the first UI layer.

Q23. What tests would you add next?

Tests for every supported label alias, malformed encodings, multiple CSV ZIPs, oversized input, empty/duplicate text, a one-row edge case, deterministic repeated training, threshold boundaries at 0.4/0.5/0.6, database limit clamping, and browser-level tests for upload, training and prediction flows.

Q24. Tell me about a trade-off you made.

I favored recall in the default model recommendation because a false negative can leave a user exposed to a scam. That intentionally accepts more false positives, but the workflow is human review, so surfacing a suspicious legitimate post is less damaging than silently missing a risky one.

Q25. What would you say if asked whether this is production-ready?

I would call it a well-structured prototype or portfolio application, not production-ready. The core flow is tested and has useful input validation, but production would require real-data validation, calibration, monitoring, security, authentication, deployment controls, privacy review and a clearer human-review process.

14. Final interview checklist

- State the problem and user before naming libraries.
- Explain the exact pipeline: fields → composed text → TF-IDF → models → metrics → full-data refit → prediction.
- Say “score,” not “guaranteed probability.”
- Explain why recall and false negatives matter.
- Be precise about what is and is not persisted.
- Admit that synthetic metrics are not production evidence.
- Mention the six automated tests and what they cover.
- Offer concrete next steps: cross-validation, calibration, threshold tuning, security and monitoring.
- Never overclaim features that are not in the repository.

Closing answer to memorize: “Postmark is a responsible, lightweight baseline for scam-language screening. Its value is not that it makes perfect fraud decisions; its value is that it creates a reproducible ML workflow, exposes the trade-offs, minimizes retained text, and keeps a human in the loop.”